SehhaTech

Multi-Tenant Clinic Management Platform

SOFTWARE ARCHITECTURE DOCUMENT

Final Project — DEPI

Repository: Final-Project-DEPI

Version 1.0

Table of Contents

1. Introduction

1.1 Purpose

This document describes the software architecture of SehhaTech, a multi-tenant clinic management platform. It is intended for developers, reviewers, and stakeholders who need to understand how the system is structured, how its components interact, and the technologies that support it.

1.2 Project Overview

SehhaTech is a SaaS (Software as a Service) platform that allows multiple independent clinics (tenants) to manage their daily operations — including reception, doctors, appointments, subscriptions, and payments — through a single shared application instance. Each clinic's data is logically isolated from the others through a multi-tenancy mechanism.

In addition to the administrative side of the platform, SehhaTech provides a dedicated Patient Portal that allows patients to browse clinics, check available appointment slots, book appointments, and manage their bookings independently.

1.3 Goals of the Architecture

- Separation of concerns between business logic, data access, and presentation.
- Support for multiple independent clinics (tenants) within a single deployment.
- Independent scalability of the administrative system and the patient-facing system.
- Maintainability and testability through interface-driven design.
- Secure authentication and integration with third-party services (payments, file storage, SMS).

2. Architectural Style

SehhaTech follows the principles of Clean Architecture (also referred to as Onion / Layered Architecture). The codebase is organized as a .NET solution composed of multiple class library and API projects, where each layer has a single, well-defined responsibility and depends only on the layer beneath it.

2.1 Layer Diagram

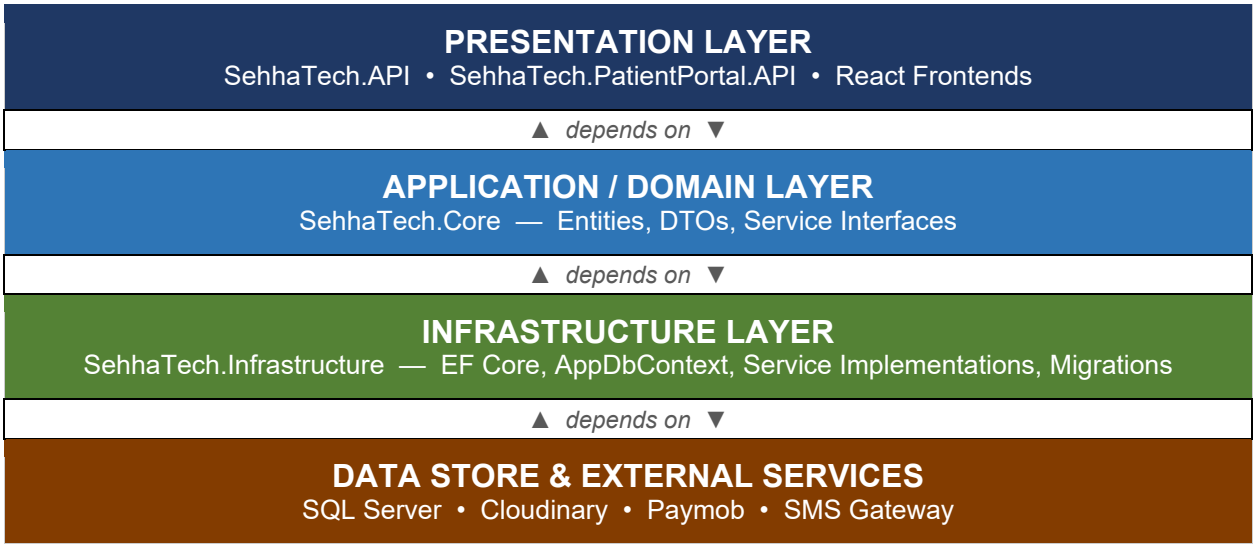

Figure 2.1 — Logical layering of the SehhaTech solution. Higher layers depend on lower layers through interfaces, never the reverse.

2.2 Dependency Rule

Dependencies always point inward: the Presentation layer depends on the Domain layer's interfaces; the Infrastructure layer implements those interfaces. The Domain layer (SehhaTech.Core) has no dependency on Infrastructure or any external framework, which keeps business rules and entity definitions independent of how data is persisted or how services are implemented.

3. Solution Structure

The solution (SehhaTech.slnx) is split into five back-end projects and two front-end applications, summarized below.

Project	Layer	Responsibility
SehhaTech.Core	Domain	Entities, DTOs, and service interfaces. Contains no implementation logic.
SehhaTech.Infrastructure	Data / Infrastructure	EF Core AppDbContext, database migrations, and concrete implementations of the Core service interfaces.
SehhaTech.API	Presentation (Admin)	Main back-office REST API consumed by clinic staff and platform administrators.
SehhaTech.PatientPortal.API	Presentation (Patient)	Dedicated REST API serving the patient-facing booking experience.
sehhatech-frontend	Presentation (Admin UI)	React (JSX) single-page application used by Super Admins, Clinic Admins, Doctors, and Reception staff.
patient-portal-frontend	Presentation (Patient UI)	React (JSX) single-page application used by patients to register, browse clinics, and book appointments.

4. Domain Layer — SehhaTech.Core

SehhaTech.Core defines the system's vocabulary: the entities that represent business data and the contracts (interfaces) that other layers must implement or consume. It contains no Entity Framework or ASP.NET Core dependencies.

4.1 Entities / Models

Entity	Description
Tenant	Represents a single clinic operating on the platform; the root of the multi-tenancy model.
User	Represents any authenticated platform user (Admin, Doctor, Reception staff, Super Admin).
Doctor	Doctor profile and association with a clinic (Tenant).
Patient	Patient record, shared between the admin system and the patient portal.
Appointment	A scheduled visit between a patient and a doctor.
Subscription	The clinic's subscription plan on the SaaS platform.
PaymentInvoice	Billing record generated for subscriptions and/or patient payments.

Additional model groups exist under Models/Portal for entities specific to the patient-facing booking flow (e.g., slots and bookings).

4.2 Service Interfaces

Interface	Purpose
IAuthService	Authentication and user session contracts.
IAdminService	Administrative operations (clinic/staff management).
IJwtService	JWT token generation and validation.
ICloudinaryService	Contract for uploading/managing files and images via Cloudinary.
IPaymobService	Contract for processing online payments via Paymob.
ISmsService	Contract for sending SMS notifications (e.g., OTP, reminders).

5. Infrastructure Layer — SehhaTech.Infrastructure

This layer provides the concrete implementations of the interfaces declared in SehhaTech.Core, along with all data-access concerns.

5.1 Data Access

- AppDbContext — the single Entity Framework Core DbContext for the application, mapping all Core entities to the SQL Server database.
- Migrations — a sequence of EF Core migrations tracks schema evolution, including dedicated migrations for patient features, the patient portal, reception workflows, and tenant-isolation fixes (e.g., FixTenantIsolation).

5.2 Service Implementations

Each interface defined in Core has a matching implementation registered in this layer via Dependency Injection, for example:

Interface (Core)	Implementation (Infrastructure)
IAuthService	AuthService
IAdminService	AdminService
IJwtService	JwtService
ICloudinaryService	CloudinaryService
—	ChatBotService (supporting in-app chatbot functionality)

5.3 Database

The platform uses Microsoft SQL Server as its relational database, accessed exclusively through EF Core via AppDbContext. Multi-tenancy is enforced at the data layer, ensuring each clinic's records (patients, appointments, invoices, etc.) remain isolated from other tenants.

6. Presentation Layer — SehhaTech.API

SehhaTech.API is the main back-office REST API. It exposes the platform's administrative capabilities and is consumed by the sehhatech-frontend application.

6.1 Controllers

Controller	Responsibility
AuthController	Login, registration, and token issuance for staff users.
SuperAdminController	Platform-wide management across all tenants (clinics).
AdminController	Clinic-level administrative operations.
DoctorController	Doctor profile and schedule management.
ReceptionController	Reception desk operations (check-in, payments, daily flow).
SubscriptionController	Management of clinic subscription plans and billing.
ChatBotController	Endpoints for the in-app assistant / chatbot feature.
UploadController	File and image uploads (backed by Cloudinary).

6.2 Middleware

TenantMiddleware intercepts incoming requests and resolves the current Tenant (clinic) context, ensuring all subsequent data operations are automatically scoped to the correct clinic.

7. Presentation Layer — SehhaTech.PatientPortal.API

This is a separate, dedicated API for patient-facing functionality. It is intentionally decoupled from SehhaTech.API so that the patient booking experience can be developed, deployed, secured, and scaled independently of the administrative back office, while still sharing the same domain entities and database through SehhaTech.Core and SehhaTech.Infrastructure.

7.1 Controllers

Controller	Responsibility
PortalAuthController	Patient registration, login, OTP verification, and password recovery.
ClinicController	Exposes the list and details of clinics available for booking.
SlotController	Returns available appointment slots for a given clinic/doctor.
BookingController	Creates and manages patient appointment bookings.

8. Front-End Applications

8.1 sehhatech-frontend (Admin & Staff Portal)

A React (JSX) single-page application consumed by SehhaTech.API. It serves multiple internal user roles through role-specific layouts and routing:

- SuperAdminLayout — platform-wide management views for the SaaS owner.
- AdminModal / Admin views — clinic-level administration.
- ReceptionSidebar / ReceptionTopbar / ReceptionToast — a dedicated workspace for reception staff.
- ProtectedRoute / PublicRoute — route guards enforcing authentication and role-based access.
- PaymobModal — embedded checkout/payment experience using Paymob.
- LanguageSwitcher — runtime switching between Arabic and English, backed by locale JSON resource files (e.g., common.json).

8.2 patient-portal-frontend (Patient Booking Portal)

A separate React (JSX) single-page application consumed by SehhaTech.PatientPortal.API, focused entirely on the patient journey:

- Landing, Login, Register, ForgotPassword, ResetPassword, VerifyOtp — full self-service authentication flow for patients.
- Clinics — browse available clinics.
- BookAppointment — select a clinic, doctor, and available slot to create a booking.
- MyBookings — view and manage existing bookings.
- Contact, Terms, Privacy — supporting static/informational pages.

9. Cross-Cutting Concerns

9.1 Multi-Tenancy

SehhaTech is a multi-tenant system: a single deployment serves many independent clinics. The Tenant entity, combined with TenantMiddleware in the API layer, ensures every request and every database query is scoped to the correct clinic. A dedicated migration (FixTenantIsolation) indicates that tenant data isolation has been an explicit, ongoing area of focus.

9.2 Authentication & Authorization

Authentication is implemented using JSON Web Tokens (JWT) via IJwtService / JwtService. Two independent authentication flows exist: one for internal staff (AuthController, used by SuperAdmin, Admin, Doctor, and Reception roles) and one for patients (PortalAuthController, including OTP-based verification).

9.3 Third-Party Integrations

Service	Purpose
Paymob	Online payment processing for subscriptions and/or patient payments.
Cloudinary	Cloud storage and delivery of uploaded images and files.
SMS Gateway (ISmsService)	Delivery of OTP codes and notification messages to patients/staff.

9.4 Internationalization (i18n)

Both front-end applications support Arabic and English through a locale-based resource system (locales/en, locales/ar) and a runtime LanguageSwitcher component, allowing the interface to adapt to the user's preferred language.

10. Example Request Flow

To illustrate how the layers cooperate, consider a patient booking an appointment through the Patient Portal:

1. The patient-portal-frontend application sends a booking request to BookingController in SehhaTech.PatientPortal.API.
2. The controller depends on a service interface defined in SehhaTech.Core (e.g., a booking/appointment service contract).
3. The concrete implementation in SehhaTech.Infrastructure executes the operation against AppDbContext, which persists the new Appointment record to SQL Server, scoped to the correct Tenant.
4. If payment is required, the implementation calls IPaymobService to initiate the transaction via Paymob.
5. A confirmation may be sent to the patient through ISmsService.
6. The result flows back through the same layers to the front-end, which updates the MyBookings view.

11. Technology Stack Summary

Concern	Technology
Back-end framework	.NET (ASP.NET Core Web API)
Architecture style	Clean Architecture (layered)
Data access	Entity Framework Core (Code-First, Migrations)
Database	Microsoft SQL Server
Authentication	JWT (JSON Web Tokens)
File / image storage	Cloudinary
Payments	Paymob
Notifications	SMS Gateway (via ISmsService)
Front-end framework	React (JSX)
Localization	JSON-based locale files (Arabic / English)
Version control / hosting	Git & GitHub

12. Conclusion

The SehhaTech platform applies Clean Architecture principles to separate domain logic, data access, and presentation concerns across distinct .NET projects, while supporting two independent front-end experiences for staff and patients. The use of multi-tenancy, interface-driven services, and third-party integrations (Paymob, Cloudinary, SMS) allows the platform to operate as a scalable, secure, multi-clinic SaaS solution.